Experiment No. 8

Aim: Developing Retrieval-Augmented Generation (RAG) Systems using Langchain

Theory:

What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is an advanced technique that combines the generative power of large language models (LLMs) with the precision of information retrieval systems. Instead of relying solely on the LLM's internal knowledge, RAG allows the model to dynamically retrieve relevant documents from an external knowledge base (e.g., vector database) and condition its responses on them. This leads to more accurate, grounded, and up-to-date outputs.

Why is RAG needed ?

To understand the necessity of Retrieval-Augmented Generation (RAG), let's consider a practical scenario at Dwarkadas J. Sanghvi College of Engineering. Imagine the college wants to deploy a smart virtual assistant for students and faculty. This assistant should be able to answer queries such as:

- "What is the exam timetable for Semester VI Computer Engineering?"
- "When is the deadline for final year project submissions?"
- "Who is the coordinator for the Data Structures lab?"
- "How many credits are required for course exemption in the DS branch?"

At first, it may seem that a powerful large language model (LLM) like GPT-3 or GPT-4 could handle these queries. However, traditional LLMs have several limitations that make them unsuitable for such academic-specific use cases:

- **Lack of College-Specific Knowledge** LLMs are trained on general-purpose data from the internet. They do not have access to college-specific information, such as internal circulars, subject timetables, lab assignments, or departmental policies. As a result, answers to student queries may be incorrect, outdated, or missing entirely.
- **Hallucinated (Fabricated) Responses** If an LLM does not know the answer, it may fabricate information, confidently giving wrong responses. For example, it might wrongly mention a faculty member as a project coordinator or generate an incorrect exam date. This kind of misinformation can lead to confusion among students and reduce trust in the system.
- **Generic Responses** Traditional LLMs tend to give generic, one-size-fits-all answers. For academic scenarios like internal project evaluation criteria or subject-specific

prerequisites, such generic replies are not useful. Students and faculty require responses that are accurate, updated, and tailored to the institution's policies.

RAG systems solve these problems by connecting the generative model to an external knowledge base—in this case, internal documents, PDFs, notices, calendars, and department-level data.

- The retriever fetches the most relevant chunks from a vector database of the college's information.
- The generative model then uses those chunks to formulate a precise, human-like response.

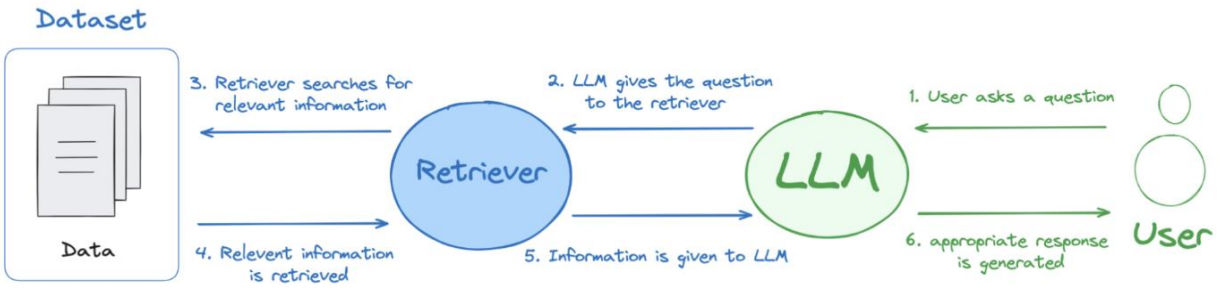


Fig 1. The most primitive architecture for retrieval-augmented generation.

1. Retrieval models:

Retrieval models are designed to fetch relevant information from a predefined corpus or knowledge base. They do not generate new content but instead return existing data that best matches the user's input. This is typically achieved through methods like **semantic search**, where both the query and the documents are transformed into **vector embeddings**, and similarity measures (like cosine similarity) are used to identify the most relevant results.

Retrieval models are especially useful for tasks that require high factual accuracy or reference to specific sources—such as searching documentation, internal reports, or domain-specific texts. Their output is grounded in reality because it comes directly from stored documents. However, they are limited in that they cannot synthesize or articulate new responses beyond what is already present in the database.

Strengths:

- High factual accuracy.
- Efficient at handling domain-specific or up-to-date content.
- Easy to update (just add new documents to the vector store).

Limitations:

- Cannot generate new language or explanations.
- Limited to the scope of the stored content.

2. Generative Models

Generative Models—such as large language models (LLMs) like GPT-4 or Claude—are trained to generate human-like text. Given a prompt, they can produce novel sentences, paragraphs, or even complete documents. These models learn patterns, grammar, and semantic relationships from massive datasets, enabling them to compose coherent and contextually relevant outputs.

Generative models excel in fluency, creativity, and adaptability. They can summarize long texts, translate between languages, answer questions, or write stories. However, they rely on patterns seen during training and do not have real-time access to external knowledge unless explicitly provided. This makes them prone to hallucination—generating plausible but factually incorrect responses.

Strengths:

- Fluent and expressive language generation.
- Capable of summarization, explanation, paraphrasing, and reasoning.
- Flexible and creative in open-ended tasks.

Limitations:

- May produce factually incorrect or hallucinated outputs. Internal knowledge may become outdated.
- Cannot reference specific documents unless manually provided.

Components of a RAG System:

1. Embedding Model - Converts raw text (queries, documents) into dense vector representations. These vectors capture semantic meaning and enable similarity search. Common choices include OpenAI embeddings, HuggingFace all-MiniLM-L6-v2, Cohere, etc.

2. Text Splitter / Chunker - Long documents are split into smaller, manageable "chunks" (e.g., 500 tokens) so that relevant segments can be retrieved efficiently. Helps ensure retrieval precision and keeps inputs within LLM context limits.

3. Vector Store / Vector Database - Stores the vector representations of document chunks. Enables similarity search (via cosine similarity or inner product) to retrieve relevant documents. Popular choices: FAISS, Chroma, Pinecone, Weaviate, Qdrant.

4. Retriever - Given a user query, generates its embedding and performs a search over the vector database to find top-k relevant chunks. The retriever does not generate answers—it only fetches context.

5. Prompt Constructor - Merges the retrieved context chunks with the user's question into a final prompt. Templates the prompt in a way that the LLM knows how to use the context effectively.

6. Generative LLM - Reads the retrieved context and generates a coherent, context-aware response. Can be any LLM such as OpenAI GPT-3.5/4, Claude, Mistral, or a local model via Hugging Face.

Exercise 01:

To build a custom Retrieval-Augmented Generation (RAG) chatbot that answers queries related to D. J. Sanghvi College of Engineering using LangChain, any LLM of your choice (e.g., LLaMA 3.1), and a user-friendly interface (optional).

Students will design a local-first RAG system that:

- Processes D. J. Sanghvi-specific data (faculty details and syllabus PDFs).
- Embeds and stores documents in a vector database.
- Retrieves relevant context and generates responses using a local LLM.
- Presents results through a custom Gradio or Streamlit-based chatbot interface.

Tasks:

Data Collection:

- **Scrape faculty data from the CSE(DS) department here. Collect name, designation, joining date to begin with.**
- **Download the syllabus PDF for the CSE(DS) department for different semesters.**
- **Clean and convert the scraped faculty information and syllabus PDF into plain text files.**

Preprocessing & Storage:

- Break the syllabus and faculty text into smaller, readable sections (chunks).
- Convert each section into a numerical format (embedding) using any model.
- Save these embeddings in a vector database, so relevant sections can be quickly retrieved later.

RAG Pipeline:

- Create a system that searches for the most relevant information based on a user's question. Use any LLM of your choice (e.g., OpenAI / Claude / LLama) to generate the final answer using the retrieved information.

Frontend Integration:

- Build a simple chatbot interface using tools like Gradio or Streamlit.
- The user enters a question → the system finds the most relevant local content → the AI generates an answer → the response is shown on the screen.

Expected Output: A working chatbot that can answer queries like:

- “Who is the HOD for CSE DS department ?”
- “What subjects are taught in the 5th semester ?”
- “What are some electives a student can choose from in the 6th semester ?”

Bonus - to be attempted once Exercise is completed (Optional)

- Enhance faculty data scraping by adding more details, such as: Area of Interest Qualifications Years of Experience List of Publications
- Expand the system to support other departments across the college using the same pipeline.
- Add support for additional student-facing queries such as: Admission process and cut-offs, Fee structure, Department achievements etc.

Implementation:

The chatbot is written in Python using LangChain. Each of the four tasks in Exercise 01 is a separate script, and the finished app is deployed on Streamlit Community Cloud.

Live app: https://lrm-rag.streamlit.app/
Source code: https://github.com/Shrutiiicodes/lrm-rag

Tools and Technologies

- **Data collection:** requests and BeautifulSoup (web scraping), pypdf (PDF to text).
- **Framework:** LangChain (documents, text splitter, vector store, prompt templates and LCEL chains).
- **Embedding model:** sentence-transformers/all-MiniLM-L6-v2 from Hugging Face, which runs locally and produces 384-dimensional vectors.
- **Vector database:** FAISS.
- **LLM:** openai/gpt-oss-120b (answers) and openai/gpt-oss-20b (query routing), both open-weight models served through the Groq API. Groq has retired its Llama 3 models, so gpt-oss is used instead.
- **Frontend and deployment:** Streamlit, hosted on Streamlit Community Cloud.

System Architecture

User question → Query router → Retriever (FAISS + metadata filters) → Prompt constructor → LLM → Answer shown in the Streamlit chat.

- **Query router:** an LLM call rewrites the question as a standalone question, using the chat history, and identifies its topic (faculty, department, syllabus or other) and the semesters it refers to.
- **Retriever:** searches the FAISS index for the most relevant chunks, restricted to the right semesters or data source.

- **Prompt constructor:** merges the retrieved chunks with the question and instructs the model to answer only from this context.
- **Generative LLM:** gpt-oss-120b writes the final answer, or says it doesn't know when the context does not contain the answer.

1. Data Collection (scrape.py)

- **Faculty:** the CSE (DS) faculty page was scraped for the name, designation and joining date of all 20 faculty members. The website mixes date formats (02.07.2007 and 01-09-2023), so all dates were normalised (e.g. 2 July 2007) and each person was written as one sentence.
- **Department information:** the department's About Us text and key points (year of establishment 2020, intake 180).
- **Syllabus:** the syllabus table on the department page was parsed to pick the latest-scheme (DJS23) PDF for Semesters III to VIII. Semesters I and II come from the common first-year DJS23 PDF on the Basic Science & Humanities page, which was split into the two semesters.
- All data was cleaned and saved as plain text files: faculty.txt, about.txt and syllabus_sem1.txt to syllabus_sem8.txt.

```
for card in soup(FACULTY_URL).select("div.shadow"):
    spans = [s.get_text(" ", strip=True) for s in card.find_all("span")]
    name, desig, joined = spans[:3]
    raw = joined.removeprefix("Joined on").strip()   # "02.07.2007" or "01-09-2023"
    d = datetime.strptime(re.sub(r"\D", ".", raw), "%d.%m.%Y")
    lines.append(f"{name} is {role} of the CSE (Data Science) department at DJSCE. "
                 f"Designation: {desig}. Joined on {d.day} {d:%B %Y}.")
```

2. Preprocessing and Storage (ingest.py)

- **Chunking:** each faculty member is one chunk. Each semester's scheme table, which lists every course, elective and credit, is kept whole as one chunk. The rest of the syllabus is split with RecursiveCharacterTextSplitter (chunk size 1200, overlap 150).
- **Metadata:** every syllabus chunk is prefixed with its semester, e.g. "[Semester 5 (5th semester, Third Year) ...]", and tagged with its semester number and kind (scheme or detail).
- **Embedding and storage:** the 633 chunks are embedded with all-MiniLM-L6-v2 and saved in a FAISS index on disk.

```
splitter = RecursiveCharacterTextSplitter(chunk_size=1200, chunk_overlap=150)
docs.append(Document(page_content=prefix + scheme_table,
                     metadata={"source": f.name, "semester": sem, "kind": "scheme"}))
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
FAISS.from_documents(docs, embeddings).save_local("store")
```

3. RAG Pipeline (rag.py)

- **Query routing:** a small LLM call with structured output (a Pydantic model) turns each question into a standalone question and returns its topic and semesters. For example, "tell me their names" after a question about new teachers is rewritten to name those teachers, and "third year, second half" is mapped to Semester 6.

- **Retrieval:** the embedding model cannot reliably tell "5th semester" from "6th semester", so the semesters become a FAISS metadata filter, and that semester's scheme table is always included. Faculty questions receive the complete faculty list, so counting questions such as "how many teachers joined before 2025?" are answered correctly. Department questions receive the About Us text.
- **Generation:** the retrieved chunks and the question are merged into a prompt that tells the model to answer only from the context and explains the course-code conventions (PE = elective, OE = open elective, L = laboratory). gpt-oss-120b generates the answer at temperature 0.

```
class Route(BaseModel):
    question: str                # standalone rewrite of the question
    topic: Literal["faculty", "department", "syllabus", "other"]
    semesters: list[int]         # e.g. "third year, second half" -> [6]

router = ROUTER | ChatGroq(model="openai/gpt-oss-20b").with_structured_output(Route)
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)
chain = PROMPT | llm | StrOutputParser()
```

4. Frontend Integration (app.py)

- A Streamlit chat interface (st.chat_input and st.chat_message). The chat history is kept in the session and passed to the query router, so follow-up questions work.
- The vector store and models are loaded once with st.cache_resource. The Groq API key is read from a .env file locally, or from Streamlit Secrets when deployed.
- The app is deployed on Streamlit Community Cloud. It installs a CPU-only build of PyTorch to stay within the free tier's limits, and redeploys automatically on every push to GitHub.

Output:

The chatbot was tested with the three expected queries from the exercise, and with indirect and follow-up questions. The answers match the department website and the syllabus PDFs. Questions outside the collected data (for example, the MBA fee structure) are answered with "I don't know" instead of a made-up answer.

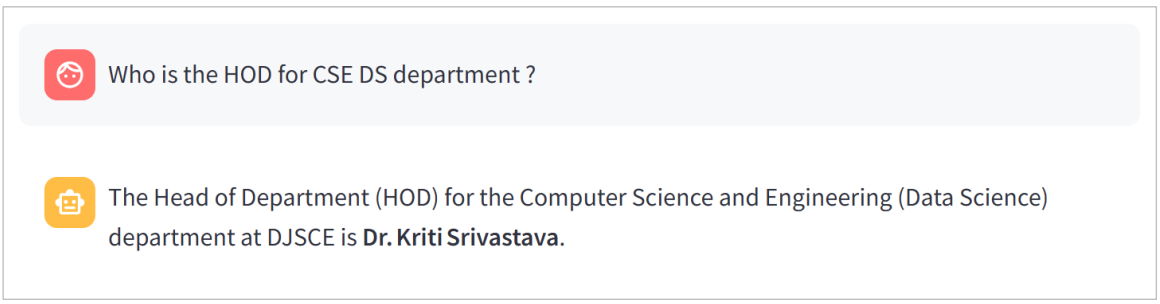


Fig 2. Expected query 1: "Who is the HOD for CSE DS department?"



Fig 3. Expected query 2: "What subjects are taught in the 5th semester?"

What are some electives a student can choose from in the 6th semester ?

Elective (Programme/Department-Elective) courses offered in Semester VI of the B-Tech CSE (Data Science) programme

Code	Theory title (elective)	Corresponding laboratory (if any)
DJS23DCPE611	Time Series Analysis	**DJS23DLPE611** – Time Series Analysis Laboratory
DJS23DCPE612	Analysis of AI Algorithms	**DJS23DLPE612** – Analysis of AI Algorithms Laboratory
DJS23DCPE613	Medical Imaging Informatics and Interoperability	**DJS23DLPE613** – Medical Imaging Informatics and Interoperability Laboratory
DJS23DCPE614	Ethical Hacking and Digital Forensics	**DJS23DLPE614** – Ethical Hacking and Digital Forensics Laboratory
DJS23DCPE616	Computer Vision	**DJS23DLPE616** – Computer Vision Laboratory
DJS23DCPE617	Robotics and AI	**DJS23DLPE617** – Robotics and AI Laboratory
DJS23DCPE618	Applied Game Theory	**DJS23DLPE618** – Applied Game Theory Laboratory
DJS23DCPE619	Information Security	**DJS23DLPE619** – Information Security Laboratory

These eight courses are the **Programme Elective (PE) / Department Elective** options for the 6th semester.
Students are required to select **any one** of these Programme Elective courses (as indicated by the "Any one Programme Elective Course (PEC) from 4@ and 5@" note).

No Open-Elective (OE) courses are listed in the provided syllabus for Semester VI.

Fig 4. Expected query 3: "What are some electives a student can choose from in the 6th semester?"

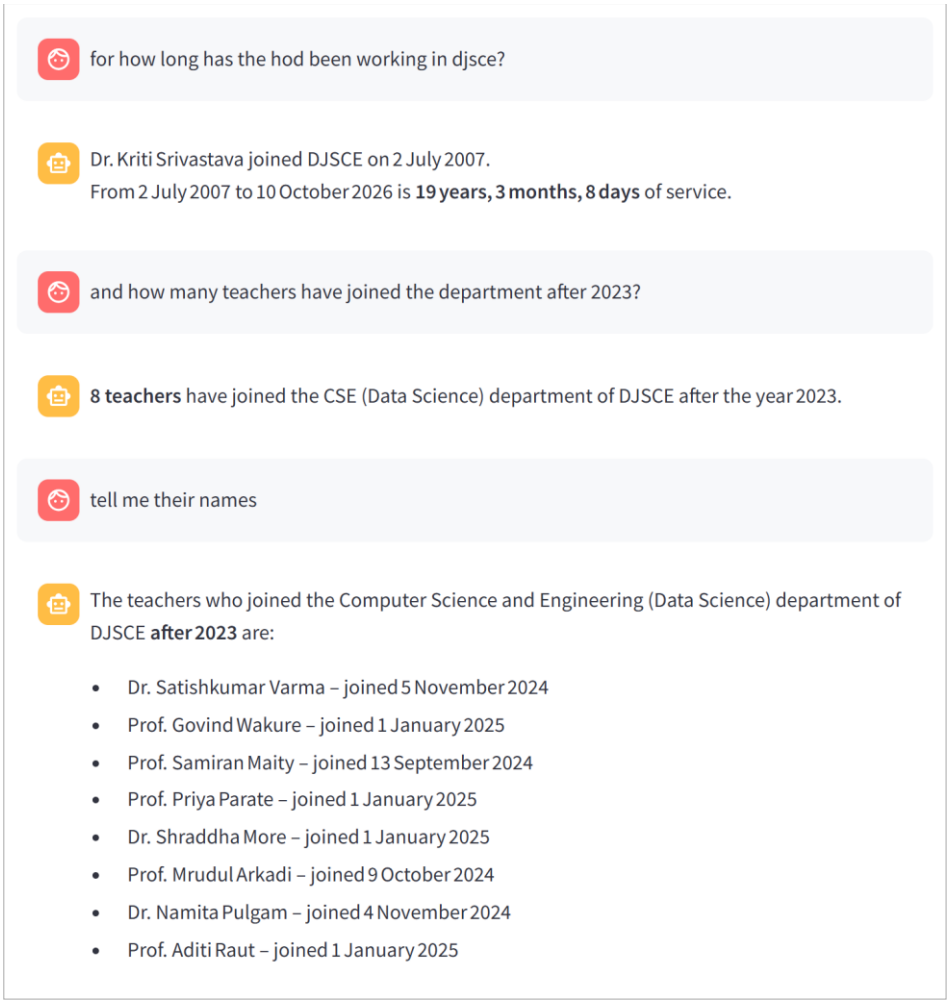


Fig 5. Indirect and follow-up questions: the HOD's length of service is calculated from the joining date, and "their" in the last question refers back to the previous answer.

Conclusion:

In this experiment, a Retrieval-Augmented Generation (RAG) chatbot for the CSE (Data Science) department of D. J. Sanghvi College of Engineering was developed using LangChain. Faculty details, department information and the DJS23 syllabus for all eight semesters were scraped, converted to text, split into chunks, embedded with all-MiniLM-L6-v2 and stored in a FAISS vector database. For every question, the most relevant chunks are retrieved and passed to an LLM, which answers only from that context. The answers are therefore grounded in the college's own data, and the chatbot says it does not know instead of hallucinating when the information is missing.

The experiment also showed the limits of plain similarity search: a small embedding model cannot tell semesters apart, and top-k retrieval cannot answer counting questions over the whole faculty list. Metadata filters, keeping the scheme tables as whole chunks, and an LLM-based query router solved these problems, and allowed the chatbot to handle indirect and follow-up questions. The final system is deployed as a public web app on Streamlit Community Cloud.